

Instituto Politécnico de Lisboa
Instituto Superior de Engenharia de Lisboa
Licenciatura em Engenharia Informática e Multimédia
Mobile Application Development
2025/2026

SET POINT APP



Student:
Mariana Amador de Almeida, 49749

Professor:
Prof. Pedro Fazenda

05th of July 2026

TABLE OF CONTENTS

EXECUTIVE SUMMARY	5
APPLICATION OVERVIEW	7
Purpose of the Application.....	7
Main Functionalities.....	8
User Authentication	8
Team Management.....	9
Player Management.....	9
Match Management.....	11
Live Match Tracking.....	11
Match Statistics	12
Season Statistics	13
Player Profiles.....	14
Typical User Workflow.....	15
ARCHITECTURE & TECHNOLOGIES	19
MVVM Architectural Pattern	19
Model Layer.....	19
View Layer	20
ViewModel Layer	21
Reactive State Management	22
Benefits of the Adopted Architecture	22
Main Technologies	23
Kotlin.....	23
Jetpack Compose	23
Jetpack Navigation Compose	24
StateFlow	24
Firebase Authentication	24
Cloud Firestore	25
Data Model and Firestore Structure	25
Firestore Collections	26
Database Relationships.....	29
Navigation Architecture	30

How AI was Used	31
Examples of AI-Assisted Prompts	31
Critical Evaluation of AI Usage	32
APPLICATION QUALITY	34
Functionality and Reliability	34
User Interface and Usability	34
LESSONS LEARNED	35
FUTURE WORK	36
CONCLUSION	38

TABLE OF FIGURES

Figure 1 - Register Screen	8
Figure 2 - Login Screen.....	8
Figure 3 - Team Profile Screen	9
Figure 4 - Team Screen	10
Figure 5 - Add Player Screen	10
Figure 6 - Finished Matches	11
Figure 7 - Scheduled Matches	11
Figure 8 - Live Matches	11
Figure 9 - Starting Lineup Screen	12
Figure 10 - Live Match Screen	12
Figure 11 - Substitution Screen.....	12
Figure 12 - Match Statistics Screen	13
Figure 13 - Match Statistics Screen	13
Figure 14 - Season Statistics Screen	14
Figure 15 - Season Statistics Screen	14
Figure 16 - Player Profile Screen	15
Figure 17 - Home Screen	16
Figure 18 - Account Created Screen	17
Figure 19 - Dashboard Screen	18
Figure 20 - MVVM Architecture	19
Figure 21 - Model Layer	20
Figure 22 - View Layer	21
Figure 23 - ViewModel Layer	22
Figure 24 - Team Data Class.....	26
Figure 25 - Player Data Class.....	27
Figure 26 - Match Data Class	27
Figure 27 - MatchLineup Data Class	28
Figure 28 - SetResults Data Class	28
Figure 29 - MatchPlayerStats Class	29
Figure 30 - Database Relationships	29

EXECUTIVE SUMMARY

SET POINT is an Android application designed to support volleyball coaches and teams in the management and analysis of their competitive matches. The application addresses the need for a centralized platform capable of combining team management, player management, live match tracking, and statistical analysis into a single mobile solution. While many existing applications focus exclusively on statistics or scorekeeping, Set Point was developed with the objective of providing coaches with an integrated tool that supports the entire workflow before, during, and after a match.

The application targets volleyball coaches, assistant coaches, team managers, and players who require an efficient way to organize their teams and collect meaningful performance data. By digitizing processes that are frequently carried out using paper notes or spreadsheets, the application improves data reliability, reduces administrative effort, and enables immediate access to player and team performance indicators.

Set Point allows coaches to create and manage their team profile, register players, schedule future matches, define the starting lineup, and record important events during a live match. Throughout the game, actions such as points, kills, aces, blocks, and errors are associated with individual players, while the application updates the match score, manages set progression according to official volleyball rules, and determines the final match result. Once a match is completed, all collected data is permanently stored and becomes available for post-match analysis.

One of the application's main strengths is its statistics module. Individual match statistics are complemented by season-wide performance indicators, allowing coaches to evaluate player consistency over time rather than relying solely on isolated performances. The application also introduces a custom Most Valuable Player (MVP) calculation based on a weighted scoring formula, rewarding positive contributions such as kills, aces, and blocks while penalizing errors. This provides a more representative evaluation of player performance than simply counting the number of points scored.

From a technical perspective, Set Point was developed entirely in Kotlin using Jetpack Compose and following the Model-View-ViewModel (MVVM) architectural pattern. Navigation between screens is handled using Jetpack Navigation Compose, while Firebase Authentication provides secure user authentication and Cloud Firestore is responsible for persistent storage of teams, players, matches, and statistical data.

Overall, the project demonstrates the development of a mobile application that integrates modern Android development practices with cloud-based services and a user-centered design approach. Beyond implementing the

required technical concepts of the Mobile Application Development course, Set Point aims to provide a practical solution for volleyball coaches by transforming match management and statistical analysis into an intuitive, efficient, and reliable mobile experience.

By combining match management, live event tracking, and performance analytics within a single mobile application, Set Point enables coaches to make more informed decisions based on objective data, ultimately contributing to a more efficient coaching process and improved team performance.

APPLICATION OVERVIEW

Purpose of the Application

Set Point was developed to provide volleyball coaches with a centralized platform capable of supporting the entire match management lifecycle. The application combines team administration, player management, match scheduling, live match tracking, and statistical analysis into a single Android application.

Traditionally, many coaches rely on handwritten notes, spreadsheets, or multiple independent applications to record player statistics and manage their teams. These approaches often make data collection time-consuming, error-prone, and difficult to analyze throughout an entire season.

The primary objective of Set Point is to digitalize these processes by offering an intuitive mobile application that allows coaches to prepare upcoming matches, record events as they occur during gameplay, and immediately access meaningful performance statistics after the match has ended. By centralizing all relevant information in a single platform, the application enables coaches to make more informed decisions based on objective performance indicators rather than subjective observations alone.

The application was designed following a user-centered approach, prioritizing ease of use during live matches while maintaining a modern interface consistent with current Android design guidelines.

Main Functionalities

The application is organized around five major functional areas that together support the complete workflow of a volleyball season.

User Authentication

The application provides secure user authentication through Firebase Authentication. New users can register using their email address or authenticate using their Google account, while returning users benefit from persistent authentication, allowing the application to remember their session between launches.

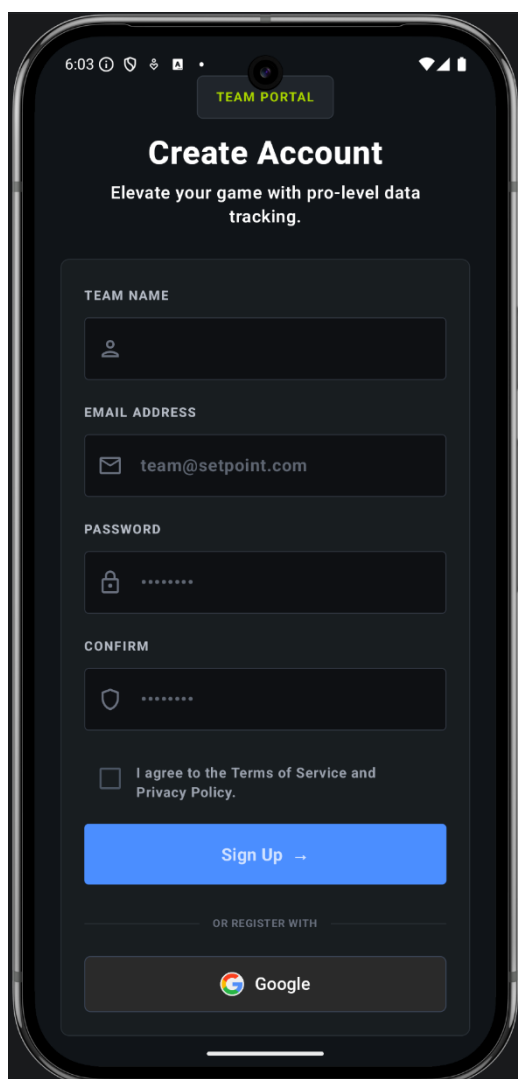


Figure 1 - Register Screen

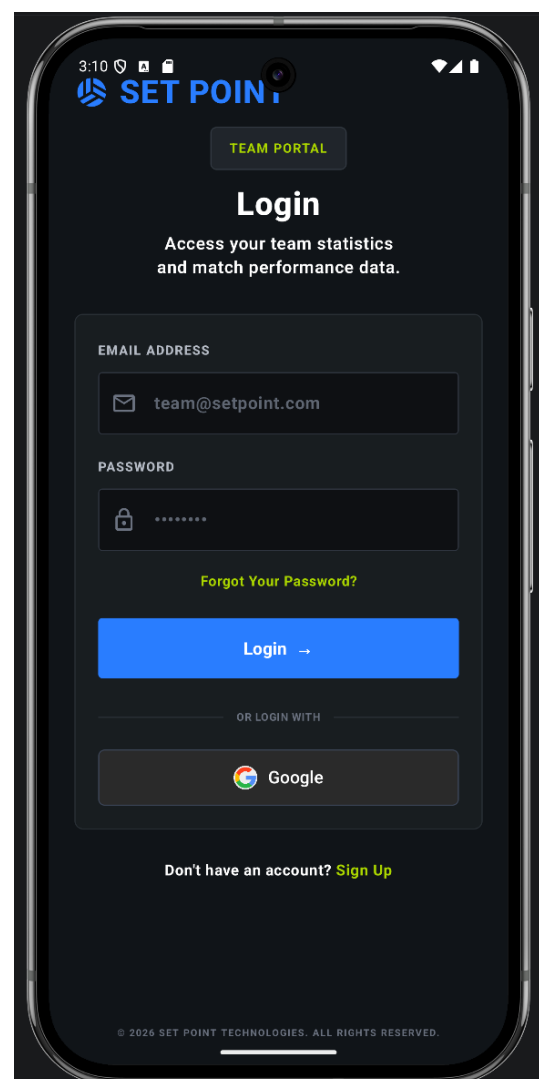


Figure 2 - Login Screen

Team Management

Each authenticated user manages a single volleyball team. Coaches can configure general team information, including the team's name, club name, category, coach name and home court. This information is stored in Cloud Firestore and can be updated at any time.

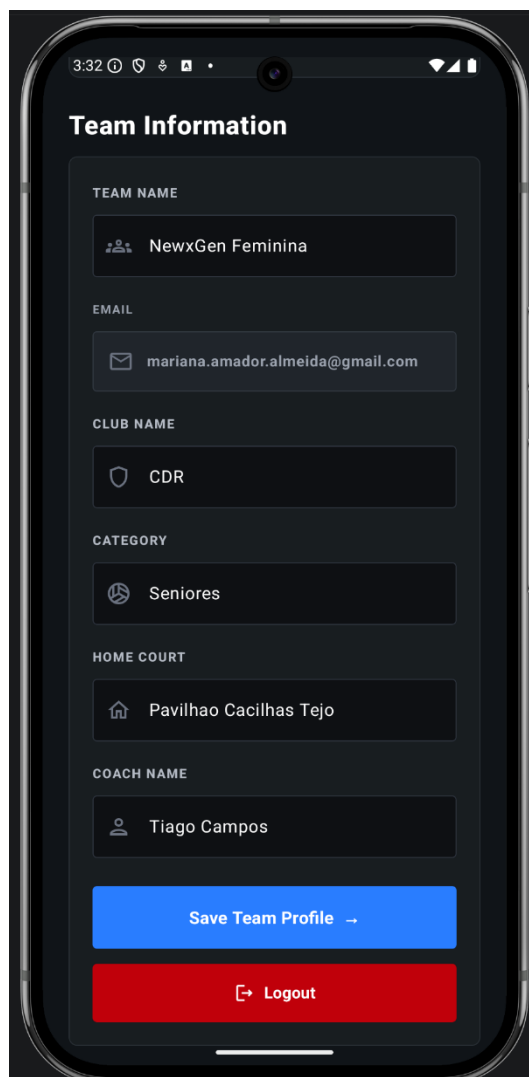


Figure 3 - Team Profile Screen

Player Management

The application allows coaches to build and maintain their team roster. Each player contains personal information such as name, jersey number, age and playing position.

Player validation mechanisms ensure data consistency, preventing duplicate jersey numbers within the same team.

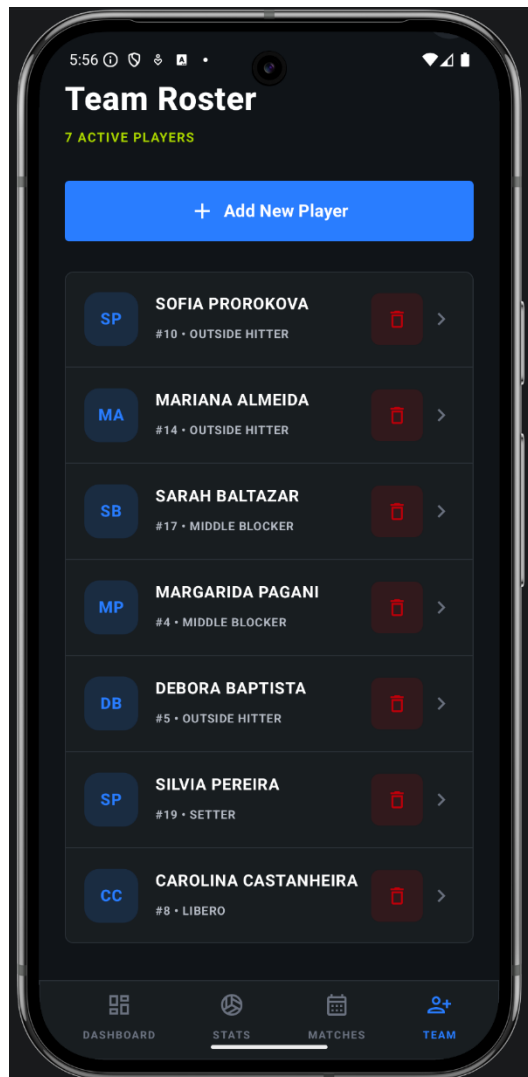


Figure 4 - Team Screen

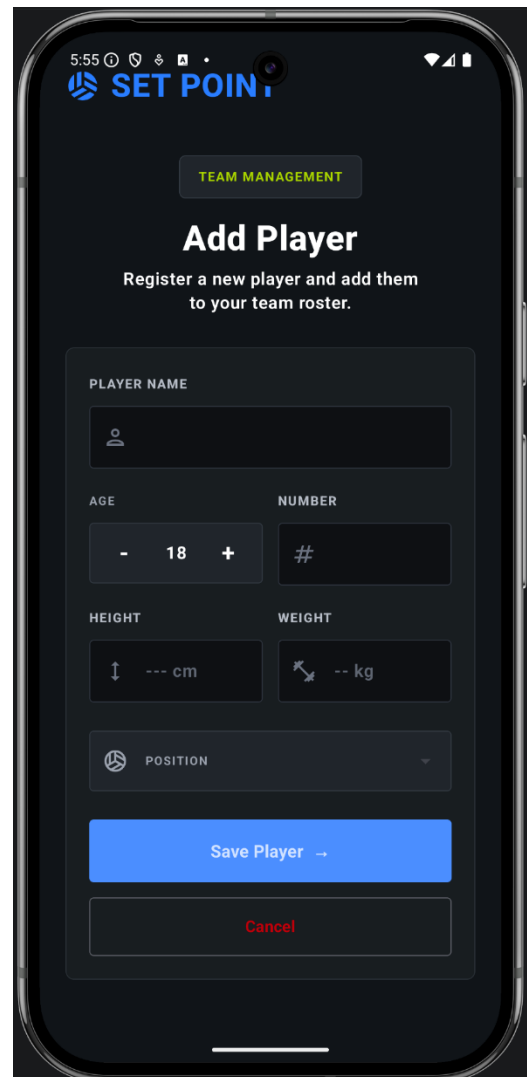


Figure 5 - Add Player Screen

Match Management

Future matches can be scheduled by specifying the opponent, date, time and location. Scheduled matches automatically become available on the Dashboard and within the Matches screen.

As the scheduled date and time are reached, the application automatically changes the match status from Scheduled to Live, allowing the coach to start tracking the game.

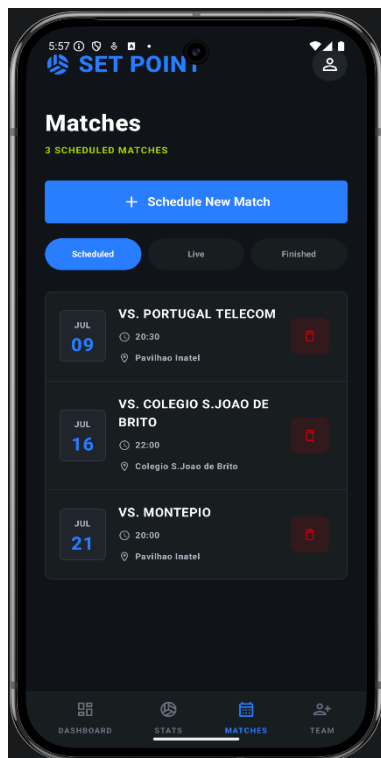


Figure 7 - Scheduled Matches

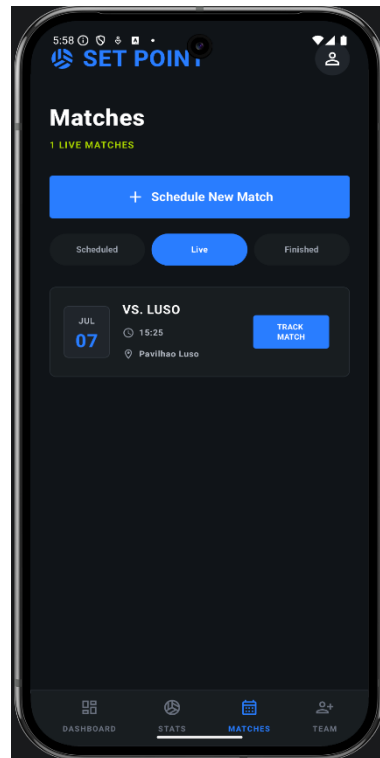


Figure 8 - Live Matches

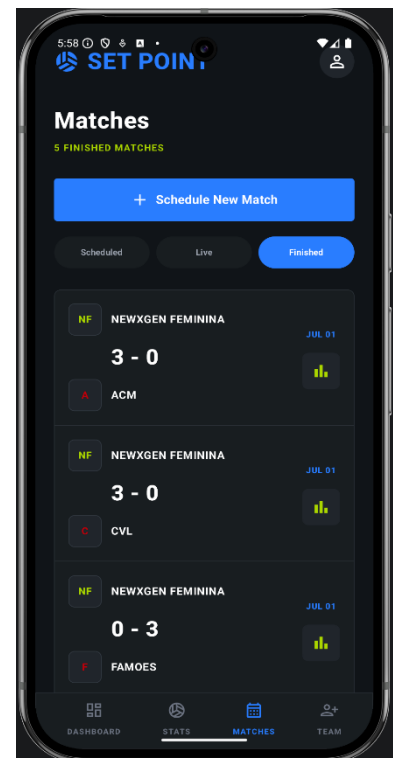


Figure 6 - Finished Matches

Live Match Tracking

During a live match, the coach first defines the starting lineup before entering the live tracking interface.

Throughout the match, players can be selected directly from the court representation and individual actions—including points, kills, aces, blocks and errors—can be recorded with a single tap. The application automatically updates both player statistics and the match score while also managing set progression according to official volleyball scoring rules.

Player substitutions can also be performed during the match without interrupting the statistics collection process.

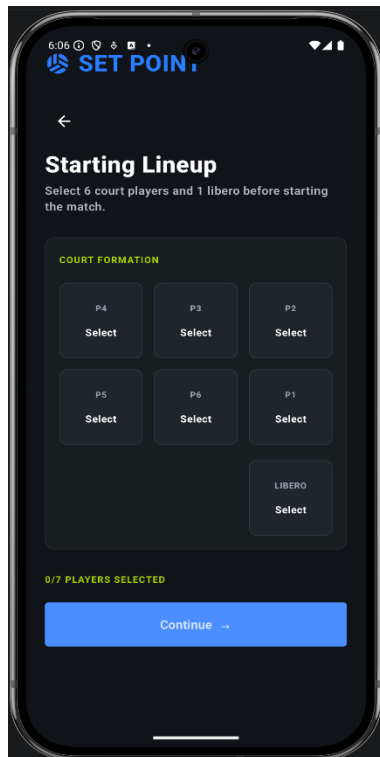


Figure 9 - Starting Lineup Screen

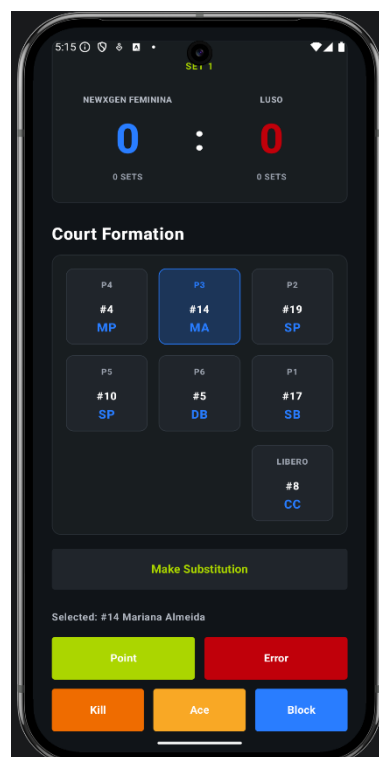


Figure 10 - Live Match Screen

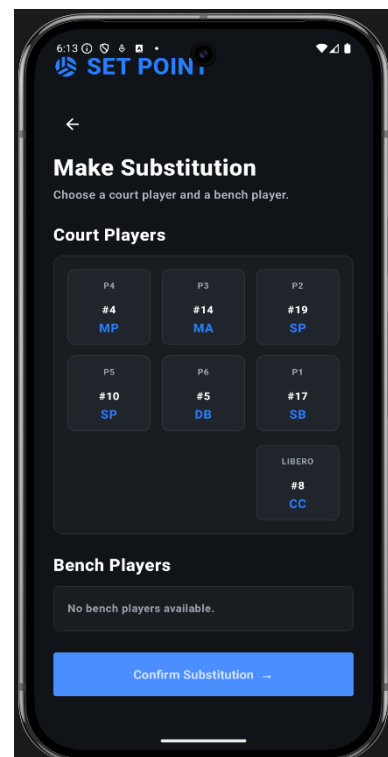


Figure 11 - Substitution Screen

Match Statistics

When a match finishes, all collected data is permanently stored. Coaches can review detailed match statistics, including:

- Final match result;
- Individual set results;
- Most Valuable Player (MVP);
- Individual player performance;

The MVP is automatically determined using a weighted scoring formula that considers both positive and negative contributions throughout the match.

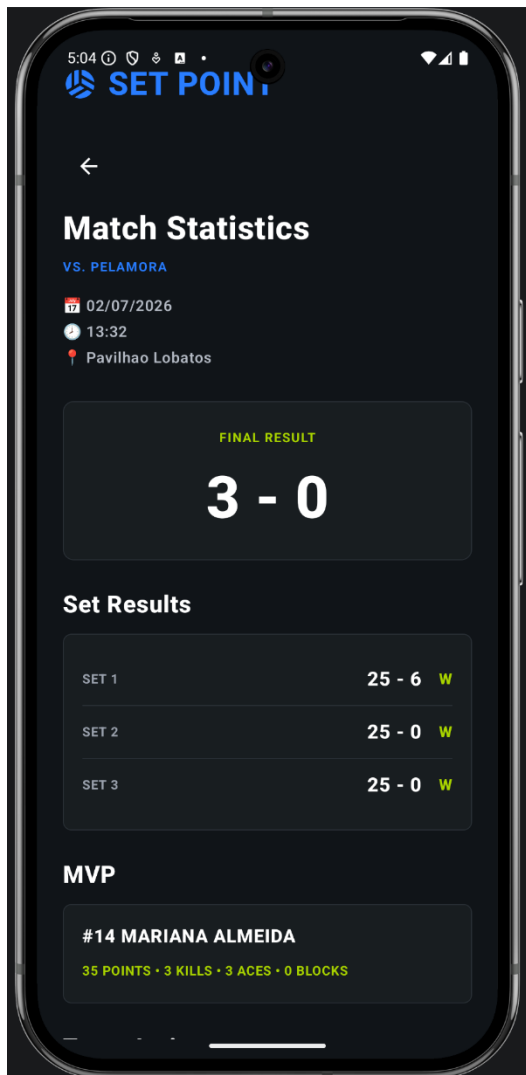


Figure 13 - Match Statistics Screen

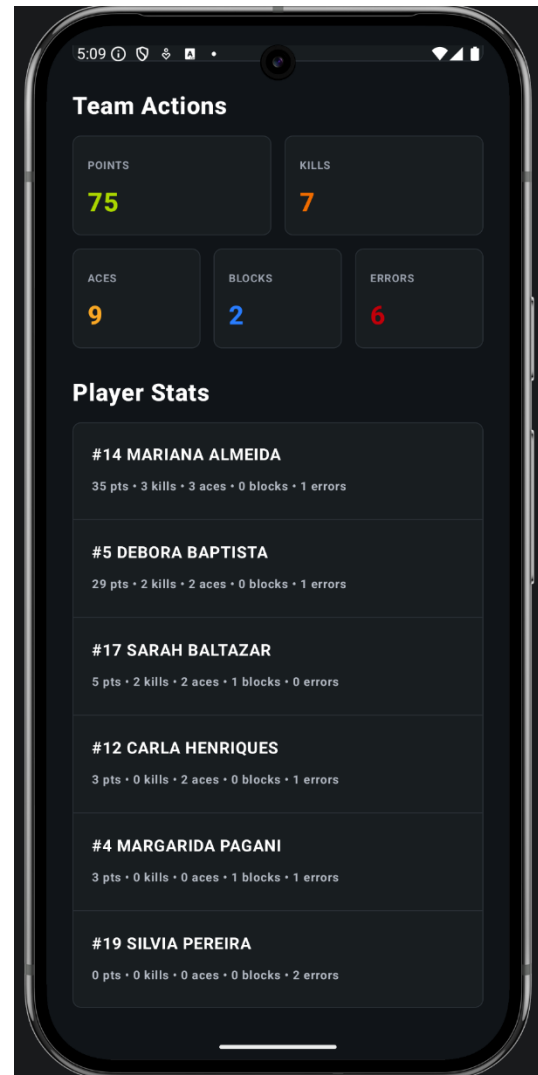


Figure 12 - Match Statistics Screen

Season Statistics

Beyond individual matches, Set Point aggregates statistics across the entire season.

The application presents leaderboards, season averages and overall performance indicators, allowing coaches to monitor player development over time and identify the strongest performers in different statistical categories.

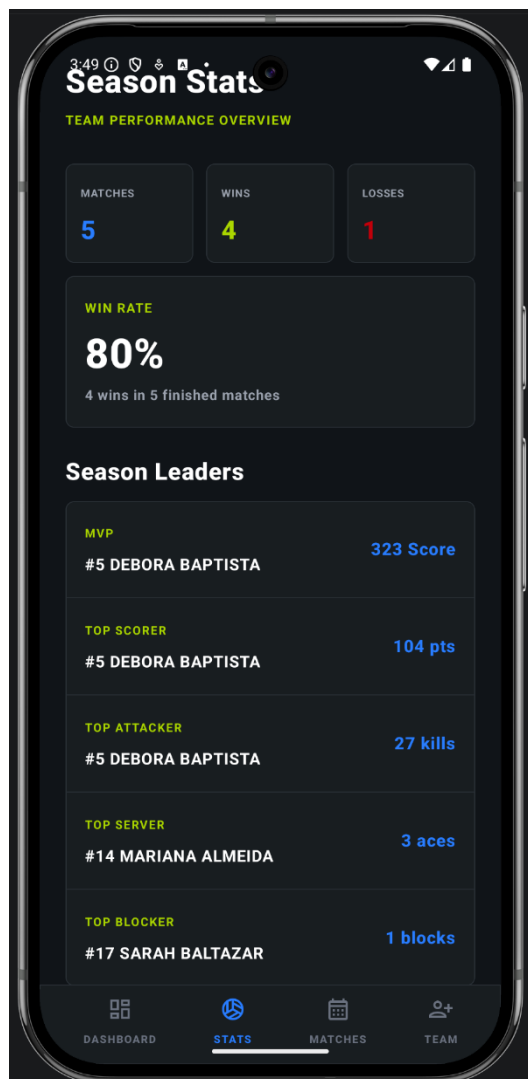


Figure 14 - Season Statistics Screen

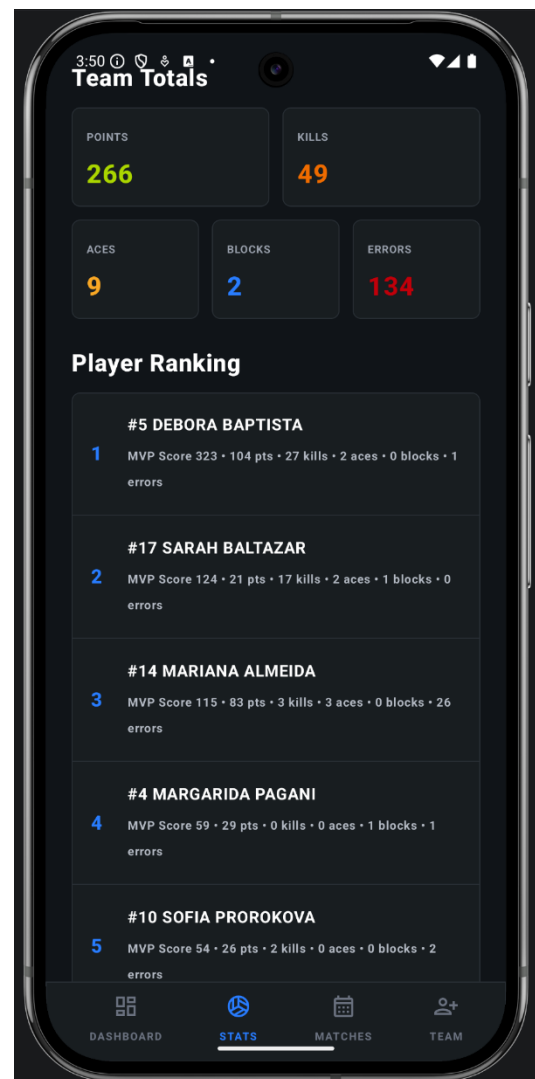


Figure 15 - Season Statistics Screen

Player Profiles

Each player has an individual profile that combines personal information with cumulative season statistics. The profile presents the player's jersey number, playing position, height, weight and age, together with performance indicators collected throughout the season, including total points, kills, aces, blocks, errors and the player's overall MVP Score. This information provides coaches with a comprehensive overview of each athlete, making it easier to monitor individual development, compare performances across the squad and identify strengths and areas for improvement.

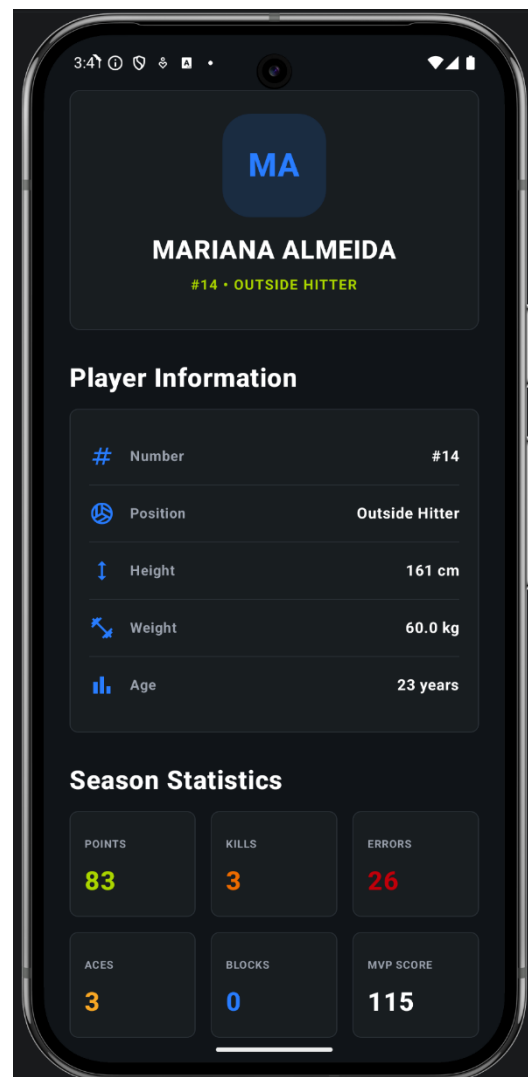


Figure 16 - Player Profile Screen

Typical User Workflow

The typical usage of Set Point follows a logical sequence that accompanies a volleyball coach throughout an entire season, from the initial authentication process to the analysis of individual player performance.

When the application is launched, users are welcomed by the Home Screen, which serves as the application's entry point. This screen presents two primary options: Log In for existing users and register for new users who wish to create an account.



Figure 17 - Home Screen

If a new user selects the registration option, they are redirected to the Register Screen, where they can create a new account by providing the required personal information. Once the registration process is successfully completed, the application displays the Account Created Screen, confirming that the account has been created successfully. Since the application requires email verification, users are informed that they must verify their email address before signing in. After completing the verification process, they can authenticate using the Login screen.

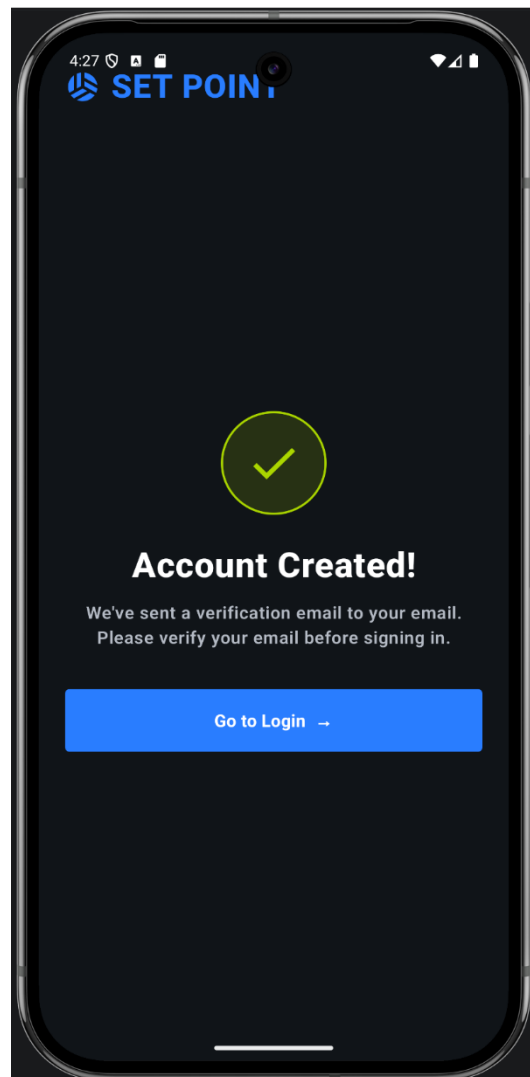


Figure 18 - Account Created Screen

Following a successful authentication, users are redirected to the Dashboard, which acts as the application's central hub. From this screen, users can quickly access the main areas of the application through the bottom navigation bar, including the Season Statistics, Matches, and Team sections. Additionally, the profile icon located in the upper-right corner provides direct access to the Team Profile.

Besides acting as the main navigation point, the Dashboard also provides an overview of the team's current season. Three summary cards present key performance indicators, including the total number of matches played, the team's win rate, and the number of registered players. Below these indicators, the Next Match section highlights the upcoming scheduled match. Once the scheduled date and time are reached, this section automatically displays a Track Match button, allowing the coach to begin live match tracking. Finally,

the Past Matches section presents the three most recent completed matches, providing quick access to their detailed statistics.

From the Dashboard, users can also navigate to the Stats, Matches and Team sections.

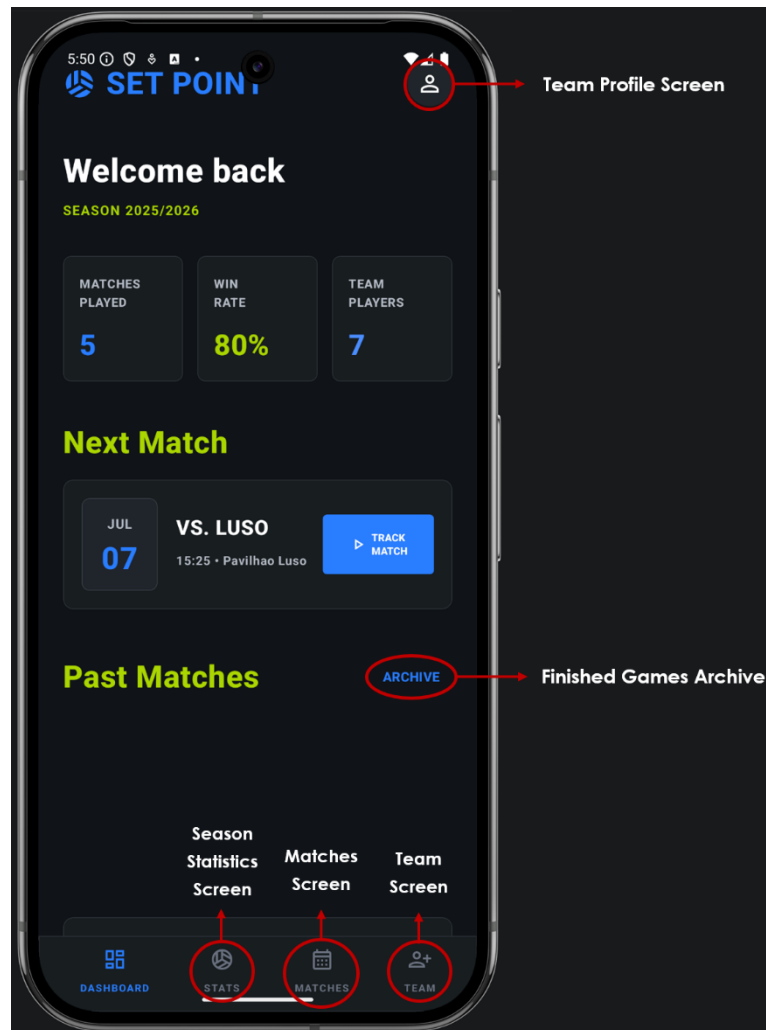


Figure 19 - Dashboard Screen

ARCHITECTURE & TECHNOLOGIES

MVVM Architectural Pattern

The Set Point application follows the Model–View–ViewModel (MVVM) architectural pattern. This architecture was selected due to its clear separation of responsibilities, improved maintainability, and seamless integration with Jetpack Compose and Android's lifecycle-aware components.

The MVVM architecture divides the application into three independent layers:

- **Model**, responsible for representing the application's data and business logic;
- **View**, responsible for displaying information and interacting with the user;
- **ViewModel**, responsible for connecting the user interface with the underlying data layer while exposing observable application state.

This separation of concerns makes the application easier to understand, maintain and extend, particularly as the project grows in complexity.

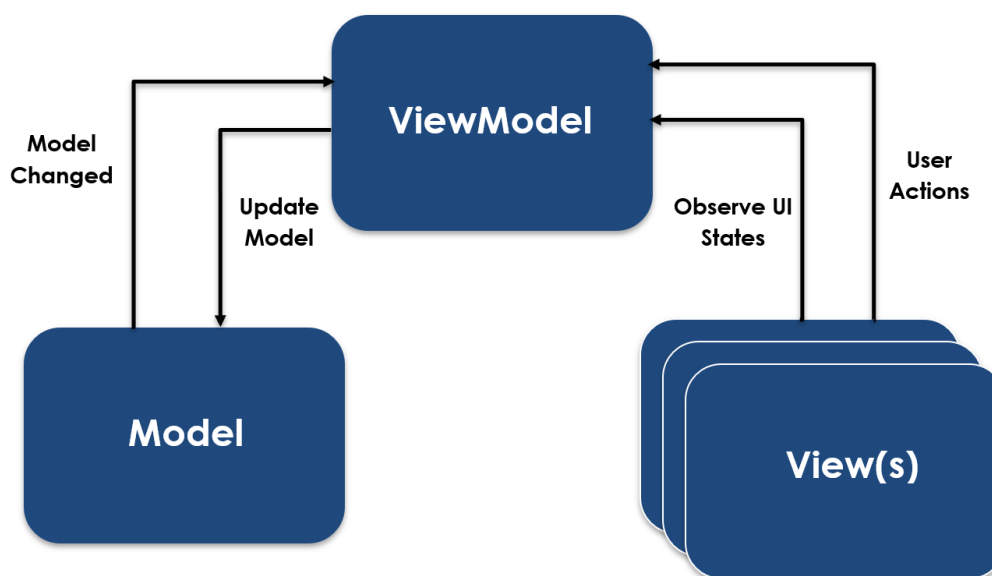


Figure 20 - MVVM Architecture

Model Layer

The Model layer represents the application's domain entities and data sources. In Set Point, this layer includes all data classes used to represent the application's information.

Persistent data storage is handled through Cloud Firestore, while user authentication is managed by Firebase Authentication. Data access is encapsulated inside dedicated repository classes, ensuring that ViewModels remain independent from the specific implementation details of Firebase.

Repositories are responsible for:

- Retrieving information from Cloud Firestore;
- Creating, updating and deleting documents;
- Synchronizing application data with the cloud;
- Abstracting Firebase APIs from the rest of the application.

This approach allows the data source to be replaced in the future without requiring modifications to the presentation layer.

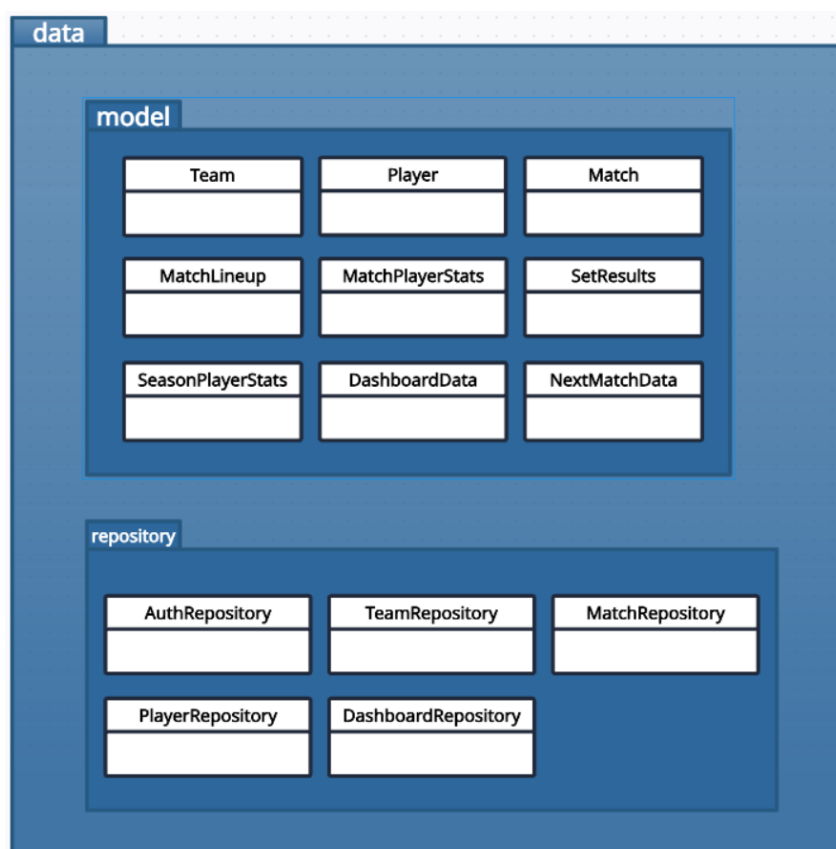


Figure 21 - Model Layer

View Layer

The View layer is implemented entirely using Jetpack Compose. Each screen within the application is represented by a dedicated composable function.

The View layer contains no business logic. Its sole responsibility is to observe the state exposed by the corresponding ViewModel and render the appropriate user interface. User interactions, such as button presses or form submissions, are forwarded to the ViewModel through callback functions, keeping the UI simple and reactive.

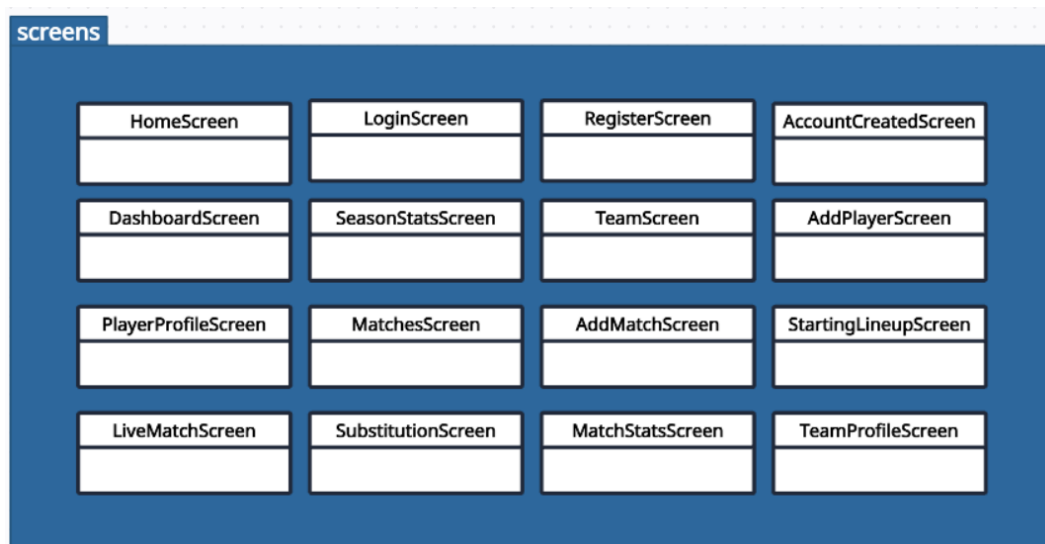


Figure 22 - View Layer

ViewModel Layer

The ViewModel layer acts as the intermediary between the user interface and the data layer.

Each major feature of the application is managed by a dedicated ViewModel.

These ViewModels expose immutable UI state objects through `StateFlow`, allowing Jetpack Compose to automatically update the interface whenever the application state changes.

Typical responsibilities of ViewModels include:

- Validating user input;
- Coordinating communication with repositories;
- Managing screen state;
- Handling loading and error conditions;
- Exposing observable data to the user interface.

By centralizing presentation logic within ViewModels, business rules remain independent from the user interface, resulting in improved code organization and easier testing.

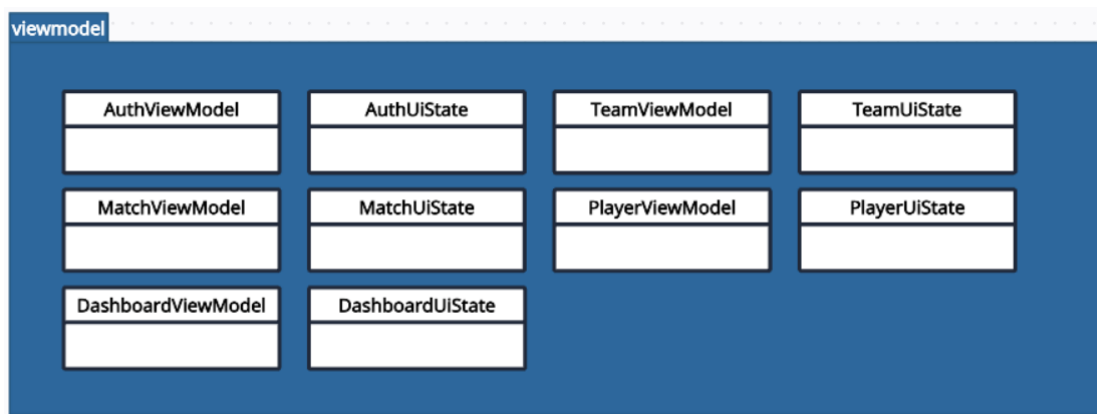


Figure 23 - ViewModel Layer

Reactive State Management

A key advantage of combining MVVM with Jetpack Compose is the reactive programming model.

Whenever application data changes, for example, when a player scores a point during a live match, the corresponding ViewModel updates its StateFlow. Since Compose observes this state, only the affected UI components are recomposed, automatically reflecting the new information without requiring manual interface updates.

This reactive behavior is heavily used throughout Set Point. Examples include:

- Automatic scoreboard updates during live matches;
- Real-time player statistics;
- Automatic dashboard refresh after a completed match;
- Dynamic season statistics;
- Automatic recalculation of MVP rankings;
- Immediate updates to player profiles after statistics change.

As a result, the user interface always remains synchronized with the underlying data while minimizing unnecessary recompositions.

Benefits of the Adopted Architecture

The adoption of the MVVM architecture provided several important advantages throughout the development of Set Point.

Firstly, the clear separation between presentation logic, business logic and data access significantly improved the organization of the project, making each component easier to understand and maintain.

Secondly, the use of repositories isolated Firebase-specific implementation details from the rest of the application. This abstraction reduces coupling and makes future modifications, such as migrating to a different backend service, Compose was particularly beneficial throughout the implementation of Set Point due to the large number of dynamic screens requiring frequent updates considerably simpler.

The reactive state management provided by StateFlow and Jetpack Compose also reduced the amount of UI code required to keep the interface synchronized with the application's state, leading to cleaner and more maintainable code.

Finally, the modular organization of the project facilitated the implementation of new features throughout development. Components such as the Dashboard, Match Statistics, Season Statistics and Player Profiles were developed independently while maintaining consistent communication through ViewModels and repositories.

Overall, the MVVM architectural pattern proved to be well suited to the requirements of Set Point, providing a scalable, maintainable and reactive foundation for the app.

Main Technologies

The development of Set Point relied on a modern Android development stack based on technologies officially recommended by Google. The selected technologies were chosen to provide a robust, scalable and maintainable architecture while ensuring a responsive user experience and seamless cloud integration.

Kotlin

The entire application was developed using Kotlin due to its concise syntax, strong type safety, null-safety mechanisms and full interoperability with existing Android libraries. Features such as data classes, extension functions, higher-order functions and coroutines significantly reduced boilerplate code while improving readability and maintainability.

Furthermore, Kotlin integrates naturally with Jetpack libraries and Firebase services, making it particularly well suited for modern Android application development.

Jetpack Compose

The user interface was implemented entirely using Jetpack Compose.

Unlike the traditional XML-based approach, Compose allows interfaces to be described directly in Kotlin code using composable functions. This declarative programming model simplifies interface development and naturally integrates with reactive state management through StateFlow.

Whenever the underlying application state changes, Compose automatically recomposes only the affected UI components, reducing unnecessary rendering while simplifying interface maintenance.

Jetpack Navigation Compose

Navigation between screens is managed using Navigation Compose.

This library provides a structured navigation graph defining all available destinations within the application.

Each feature is represented by its own screen and navigation route, allowing clear separation between different functional areas of the application.

Navigation Compose also simplifies parameter passing between screens, enabling features such as:

- Opening a specific player profile;
- Displaying a selected match;
- Navigating to match statistics;
- Opening live match tracking for a specific game.

This approach results in a scalable navigation architecture that remains easy to maintain as additional screens are introduced.

StateFlow

Reactive state management is implemented using Kotlin StateFlow.

Each ViewModel exposes immutable UI state objects that are observed directly by Jetpack Compose.

Whenever application data changes, StateFlow automatically notifies the corresponding composable functions, causing only the affected components to be recomposed.

The reactive programming model significantly reduces manual UI updates and contributes to a more responsive user experience.

Firestore Authentication

User authentication is handled through Firebase Authentication.

The application supports both:

- Email and Password authentication;

- Google Sign-In.

Firebase Authentication also provides persistent user sessions, allowing previously authenticated users to remain logged in across application restarts. Additionally, email verification is required during the registration process, ensuring that newly created accounts are associated with valid email addresses before access to the application is granted.

Using Firebase Authentication eliminated the need to implement a custom authentication system while benefiting from Google's secure authentication infrastructure.

Cloud Firestore

Persistent application data is stored using Cloud Firestore, Google's NoSQL cloud database.

Firestore stores all information required by the application, including:

- Team information;
- Player information;
- Scheduled matches;
- Live match data;
- Starting lineups;
- Match statistics;
- Season statistics.

Its document-based structure aligns naturally with Kotlin data classes and simplifies the serialization and deserialization of application data.

Cloud Firestore also offers automatic cloud synchronization, allowing all information to remain available across different devices after user authentication.

Data Model and Firestore Structure

The Set Point application relies on Cloud Firestore as its primary persistent storage solution. Firestore is a cloud-hosted NoSQL document database that stores information in collections of JSON-like documents. This document-oriented approach was particularly well suited to the application's requirements, as it closely matches Kotlin data classes while providing excellent scalability and seamless integration with Firebase Authentication.

Unlike relational databases, Firestore does not require rigid table structures or foreign keys. Instead, related information is represented through document identifiers, allowing the application to efficiently retrieve and update data

while maintaining a flexible schema capable of evolving throughout the application's lifecycle.

Each authenticated user owns an independent dataset containing the information associated with their volleyball team. This ensures complete isolation between users while simplifying data access and security management.

Firestore Collections

The application's database is organized into three main collections representing the core entities of the system.

Teams Collection

The Team collection stores the general information associated with the volleyball team managed by the authenticated coach.

```
data class Team(  
    val id: String = "",  
    val teamName: String = "",  
    val email: String = "",  
    val clubName: String = "",  
    val category: String = "",  
    val season: String = "",  
    val homeCourt: String = "",  
    val coachName: String = ""  
)
```

Figure 24 – Team Data Class

Players Collection

The Players collection stores all registered players belonging to the authenticated team.

```
data class Player(  
    val id: String = "",  
    val name: String = "",  
    val age: Int = 0,  
    val height: Int = 0,  
    val weight: Double = 0.0,  
    val position: String = "",  
    val number: Int = 0,  
    val photoUrl: String = ""  
)
```

Figure 25 - Player Data Class

Matches Collection

The Matches collection stores all scheduled, live and completed volleyball matches.

Additionally, each match stores more complex structures representing the match progression.

These include:

- Starting lineup;
- Set results;
- Individual player statistics for that match.

Keeping all match-related information within a single document simplifies retrieval operations, as the application can obtain every detail required to display the Live Match and Match Statistics screens through a single database read.

```
data class Match(  
    val id: String = "",  
    val opponent: String = "",  
    val date: String = "",  
    val time: String = "",  
    val location: String = "",  
    val status: String = "",  
    val lineup: MatchLineup = MatchLineup(),  
    val currentSet: Int = 1,  
    val teamScore: Int = 0,  
    val opponentScore: Int = 0,  
    val teamSetsWon: Int = 0,  
    val opponentSetsWon: Int = 0,  
    val setResults: List<SetResults> = emptyList()  
)
```

Figure 26 - Match Data Class

Embedded Data Structures

Rather than creating additional collections for every entity, Set Point stores several related objects directly inside Match documents.

MatchLineup

Stores the IDs of the six starting players together with the libero selected before the beginning of the match.

```
data class MatchLineup(  
    val position1PlayerId: String = "",  
    val position2PlayerId: String = "",  
    val position3PlayerId: String = "",  
    val position4PlayerId: String = "",  
    val position5PlayerId: String = "",  
    val position6PlayerId: String = "",  
    val liberoPlayerId: String = ""  
)
```

Figure 27 - MatchLineup Data Class

SetResults

Records the final score of each completed set, allowing the Match Statistics screen to display the complete progression of the match.

```
data class SetResults(  
    val setNumber: Int = 1,  
    val teamScore: Int = 0,  
    val opponentScore: Int = 0,  
    val winner: String = ""  
)
```

Figure 28 - SetResults Data Class

MatchPlayerStats

Stores the statistics achieved by each player during that specific match, including:

- Points;
- Kills;
- Aces;
- Blocks;
- Errors.

This information is later used to calculate the Match MVP and generate detailed post-match statistics.

Embedding these structures within the Match document reduces the number of required database queries and naturally groups together information that always belongs to the same match.

```
data class MatchPlayerStats(  
    val playerId: String = "",  
    val playerName: String = "",  
    val playerNumber: Int = 0,  
    val points: Int = 0,  
    val errors: Int = 0,  
    val kills: Int = 0,  
    val aces: Int = 0,  
    val blocks: Int = 0  
)
```

Figure 29 - MatchPlayerStats Class

Database Relationships

Although Cloud Firestore is a non-relational database, logical relationships still exist between the application's entities:

- Each Team owns multiple Players and multiple Matches;
- Each Match references the participating players through their unique identifiers contained within the MatchLineup and MatchPlayerStats objects.

This design avoids unnecessary data duplication while ensuring that player information can be retrieved whenever detailed statistics need to be displayed.

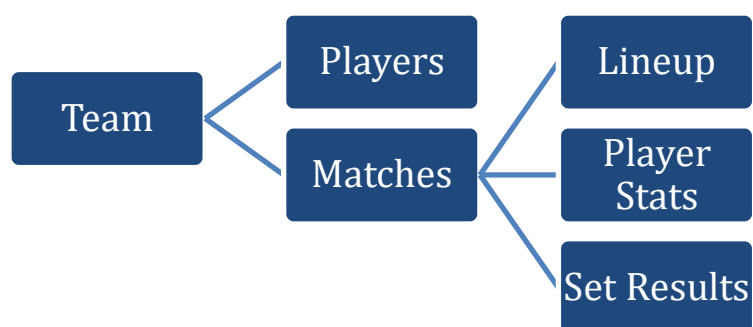


Figure 30 - Database Relationships

Navigation Architecture

The navigation within Set Point is implemented using Jetpack Navigation Compose, Google's recommended navigation framework for applications developed with Jetpack Compose. This library provides a centralized navigation graph responsible for defining all application destinations and the possible transitions between them.

The adoption of Navigation Compose simplified the management of screen transitions while maintaining a clear separation between navigation logic and the user interface. Each screen is associated with a unique route, and whenever necessary, parameters are passed through the navigation graph to identify the resources that should be displayed, such as a specific player or match.

USE OF ARTIFICIAL INTELLIGENCE

Throughout the development of Set Point, Artificial Intelligence was used as a supporting tool to assist with software development, interface design and technical problem solving. The primary AI platform used during the project was ChatGPT.

Rather than generating the application automatically, AI was used as an interactive assistant capable of discussing architectural decisions, explaining Android development concepts, suggesting improvements, identifying programming errors and accelerating the implementation of individual features.

The final responsibility for analyzing, adapting and integrating the generated suggestions always remained with the developer.

How AI was Used

AI was employed during several stages of the development process, ranging from the initial planning of the application to the implementation of advanced features.

During the design phase, AI assisted in refining the application's architecture by discussing the suitability of the MVVM pattern, the organization of repositories and ViewModels, the modelling of Firestore collections and the navigation structure implemented using Navigation Compose.

As development progressed, AI was frequently used to explain Android-specific concepts, suggest implementations using Jetpack Compose, and assist in solving programming issues encountered during integration with Firebase Authentication, Cloud Firestore and Navigation Compose.

AI also contributed to improving the user interface by proposing alternative layouts, refining screen organization, suggesting color schemes consistent with the application's visual identity and discussing usability improvements.

Examples of AI-Assisted Prompts

The following examples illustrate some of the prompts used throughout the development of the project.

Example 1 – Architecture Design

"Considering the current structure of my Android application, would the MVVM architecture be appropriate for this project?"

This discussion contributed to the definition of the application's overall architecture and helped establish a clear separation between presentation logic, business logic and data persistence.

Example 2 – User Interface Design

"Redesign the Dashboard so that it presents the most relevant information for a volleyball coach while maintaining a clean and modern interface."

The suggestions generated served as inspiration for several interface refinements, which were later adapted and implemented manually according to the specific requirements of the application.

Example 3 – Statistical Analysis

"Help design a mathematical formula capable of selecting the Most Valuable Player (MVP) by assigning different weights to positive and negative volleyball actions."

This discussion led to the definition of the weighted MVP scoring model adopted by the application.

Critical Evaluation of AI Usage

Although AI significantly accelerated the development process, its suggestions were not accepted uncritically.

Generated code and technical recommendations were carefully reviewed before being incorporated into the project. In several situations, AI proposed implementations that were either incompatible with the existing architecture, inconsistent with the application's requirements or based on assumptions that did not accurately reflect the project.

Consequently, every generated solution was manually analyzed, adapted when necessary and thoroughly tested before being integrated into the final application.

The use of AI proved particularly valuable for:

- Discussing architectural alternatives;
- Explaining unfamiliar Android concepts;
- Identifying the causes of programming errors;
- Accelerating the implementation of repetitive code;
- Refining user interface design;
- Improving technical documentation.

However, the overall design decisions, feature implementation, debugging process and final validation remained entirely under the developer's responsibility.

Overall, AI functioned as an intelligent development assistant rather than an automatic code generator, complementing the software development process while maintaining full human oversight over the final implementation.

APPLICATION QUALITY

Functionality and Reliability

Throughout the development of Set Point, particular attention was given to ensuring that all implemented features behaved consistently and reliably. Every major functionality was manually tested after implementation, including user authentication, player management, team configuration, match scheduling, live match tracking and statistical analysis.

Special care was taken when validating workflows involving multiple application components. For example, the transition between scheduled, live and finished matches was extensively tested to ensure that match status updates, navigation behavior and statistical calculations remained consistent throughout the application's lifecycle.

Input validation mechanisms were also implemented to prevent inconsistent data from being stored. Examples include mandatory form fields, duplicate jersey number validation when registering new players, email verification during user registration, and controlled input components such as date and time pickers.

These validation mechanisms improve the overall robustness of the application while reducing the likelihood of user errors.

User Interface and Usability

The application's interface was designed following the principles of Material Design 3, with a strong focus on clarity, consistency and ease of use.

The overall navigation structure was intentionally kept simple, allowing users to access the application's main features through a persistent bottom navigation bar. Frequently used actions, such as starting a live match or viewing recent match statistics, are directly accessible from the Dashboard, reducing the number of interactions required during normal use.

LESSONS LEARNED

The development of Set Point represented a valuable learning experience that combined mobile application development, cloud-based services and modern Android architecture.

One of the most significant lessons learned was the importance of designing a solid software architecture before implementing application features. Adopting the MVVM architectural pattern from the beginning greatly simplified the integration of new functionality throughout the project, allowing business logic, user interface and data persistence to evolve independently.

The project also provided practical experience with Jetpack Compose, reinforcing the advantages of declarative user interface development and reactive state management through StateFlow. Understanding how recomposition works and how application state propagates through the interface proved essential when implementing dynamic features such as live match tracking and real-time statistics.

Working with Firebase Authentication and Cloud Firestore also offered valuable insight into cloud-based mobile development. The project required careful consideration of data modelling, user authentication and efficient communication with cloud services while maintaining a responsive user experience.

Another important lesson involved the use of Artificial Intelligence as a software development assistant. Rather than replacing programming knowledge, AI proved most effective when used to discuss architectural decisions, explain unfamiliar concepts, identify possible causes of implementation issues and propose alternative solutions. This experience reinforced the importance of critically evaluating AI-generated suggestions before incorporating them into a software project.

Overall, the project significantly strengthened both technical knowledge and software engineering skills, particularly in relation to Android development, software architecture and cloud integration.

FUTURE WORK

Although Set Point already provides a comprehensive solution for volleyball team management and statistical analysis, several enhancements could further increase its usefulness and provide coaches with more detailed insights into both team and individual performance.

One possible improvement would be the integration of push notifications to remind users of upcoming matches and other important events. Timely notifications would help coaches stay informed about scheduled fixtures and improve the overall user experience by reducing the risk of missing important activities.

The Live Match Tracking module could also be extended with additional statistical categories. While the current version focuses on offensive actions such as points, kills, aces, blocks and errors, future versions could incorporate more detailed performance metrics related to serve reception, defense and setting. These aspects play a fundamental role in volleyball performance but require a more specialized understanding of the sport. Consequently, the design and implementation of these statistics should be carried out in collaboration with experienced volleyball coaches or technical specialists to ensure that the recorded metrics are both meaningful and useful for performance analysis.

Another valuable enhancement would be the introduction of a match timer capable of recording the duration of the game in real time. In addition to displaying the elapsed time during live match tracking, the application could automatically store the total match duration as well as the duration of each individual set. This information would provide coaches with additional context when analyzing player performance and match intensity.

The application could also benefit from improved support for different Android devices. Although the current interface adapts well to standard smartphone screens, future versions could introduce device-specific layouts, particularly for tablets and larger displays. A larger interface would allow more information to be presented simultaneously during live match tracking, improving usability in coaching environments.

Finally, the statistics module could be expanded by offering multiple forms of data visualization. In addition to the existing numerical summaries, future versions could include interactive charts, graphical performance trends and automatically generated PDF reports. These features would enable coaches to analyze player and team performance more effectively while also simplifying the sharing of statistical reports with players, coaching staff and club management.

Overall, these enhancements would build upon the modular architecture adopted throughout the project while preserving the application's existing design principles. By extending the Live Match Tracking system, improving statistical visualization and adapting the interface to a wider range of devices, future versions of Set Point could evolve into an even more complete tool for volleyball performance analysis and team management.

CONCLUSION

The objective of this project was to design and develop a mobile application capable of supporting volleyball coaches in the management of teams, live match tracking and statistical analysis.

The resulting application, Set Point, successfully integrates user authentication, team management, player management, match scheduling, live statistics collection and season performance analysis within a single Android application. Throughout the development process, modern Android technologies such as Kotlin, Jetpack Compose, Firebase Authentication and Cloud Firestore were combined within an MVVM architecture to produce a scalable, maintainable and responsive solution.

Beyond achieving the functional objectives initially defined, the project provided valuable practical experience in software architecture, cloud-based mobile development and reactive user interface design. The knowledge acquired throughout the project extends beyond the implementation of a single application and contributes significantly to future software engineering projects.

Overall, Set Point demonstrates how modern Android development practices can be applied to create a practical solution capable of simplifying volleyball team management while providing coaches with objective performance data to support decision-making.